ACCENTURE INNOVATION CHALLENGE 2026 . ROUND 2 . TRACK 1

ControlPlane

Repository README

An assurance gateway that decides how much checking each AI answer is worth, and proves what it decided. This document is the repository README, rendered for upload. Source github.com/Advait-Pathak9222/BurningShadows

---

An assurance gateway that decides how much checking each AI answer is worth, and proves what it decided. ControlPlane sits between your users and whatever model answers them. For every answer it estimates what the damage would be if that answer were wrong, then buys checking only where the damage prevented is worth more than the check costs. A safety floor that the budget cannot override sits underneath, and every decision is written into a tamper evident ledger.

Problem

Enterprises now put language models in front of customers, staff and money. Something has to check what those models say before it reaches a person or triggers an action. Today there are two ways to do that, and both waste money.

Check everything. Run a strong checker on every answer. This is thorough and it is expensive. In our evaluation the strongest automated check costs 160 times the cheapest one and adds 225 times the delay. Applied to all traffic it can cost more than generating the answer did.

Check a fixed sample. Check one answer in ten, or one in twenty. This is cheap and it is blind. It spends the same effort on someone asking about opening hours as on an instruction to move money, because it cannot tell the two apart.

Neither approach asks the question a business actually cares about, which is what it would cost if this particular answer were wrong. A wrong opening time is an apology. A wrong payment instruction is a loss, a regulator and an incident review.

There is a second cost that most tools ignore. Whatever the automated checks escalate, a person has to look at. On our configured prices one completed human review costs ₹120 while the most expensive automated check costs ₹3.20, so a review is 37.5 times an automated check. Measured across the budgets worth running, **81 to 98 paise in every assurance rupee is human time rather than compute**. Optimising only the compute optimises the small part of the bill.

Solution

ControlPlane treats verification as a spending decision rather than a fixed rule.

Risk and consequence. Detectors score each answer on five harm axes. Calibration turns those raw scores into probabilities. Each route carries a table of what each kind of harm costs in rupees, so risk multiplied by consequence gives an expected loss for this specific answer.

A safety floor that cannot be priced away. Each route has a release threshold selected on held out data so that the rate of harm escaping unchecked stays under a stated bound. When an answer sits above that threshold a check is mandatory no matter what the budget says.

A budget that allocates rather than rations. Below the floor, the allocator compares tiers. It buys a check when the expected loss it would prevent beats the cost of running it, adjusted by a shadow price that rises when spending runs ahead of budget. Cheap tiers handle most traffic and the expensive judge is bought only where it pays.

Human review as a scarce resource. Whatever the system will not decide alone goes to a queue that serves by expected loss per reviewer minute against the deadline, not first in first out.

Effects gated separately from words. Text can stream while checking runs. Actions that move money or change records wait behind their own gate, because words can be retracted and a payment cannot.

Everything recorded. Each decision, its inputs, its price and the policy version go into a hash chained ledger where any later edit breaks every hash after it.

How one request flows

A request arrives on a route such as `finops-agent`. The route and the country resolve a policy, which fixes what a mistake costs, how long a reviewer has and how much the route may spend.

A cheap rules pass reads the prompt on its own, before the model runs. If it looks like an attack the request is refused there and pays for neither generation nor checking.

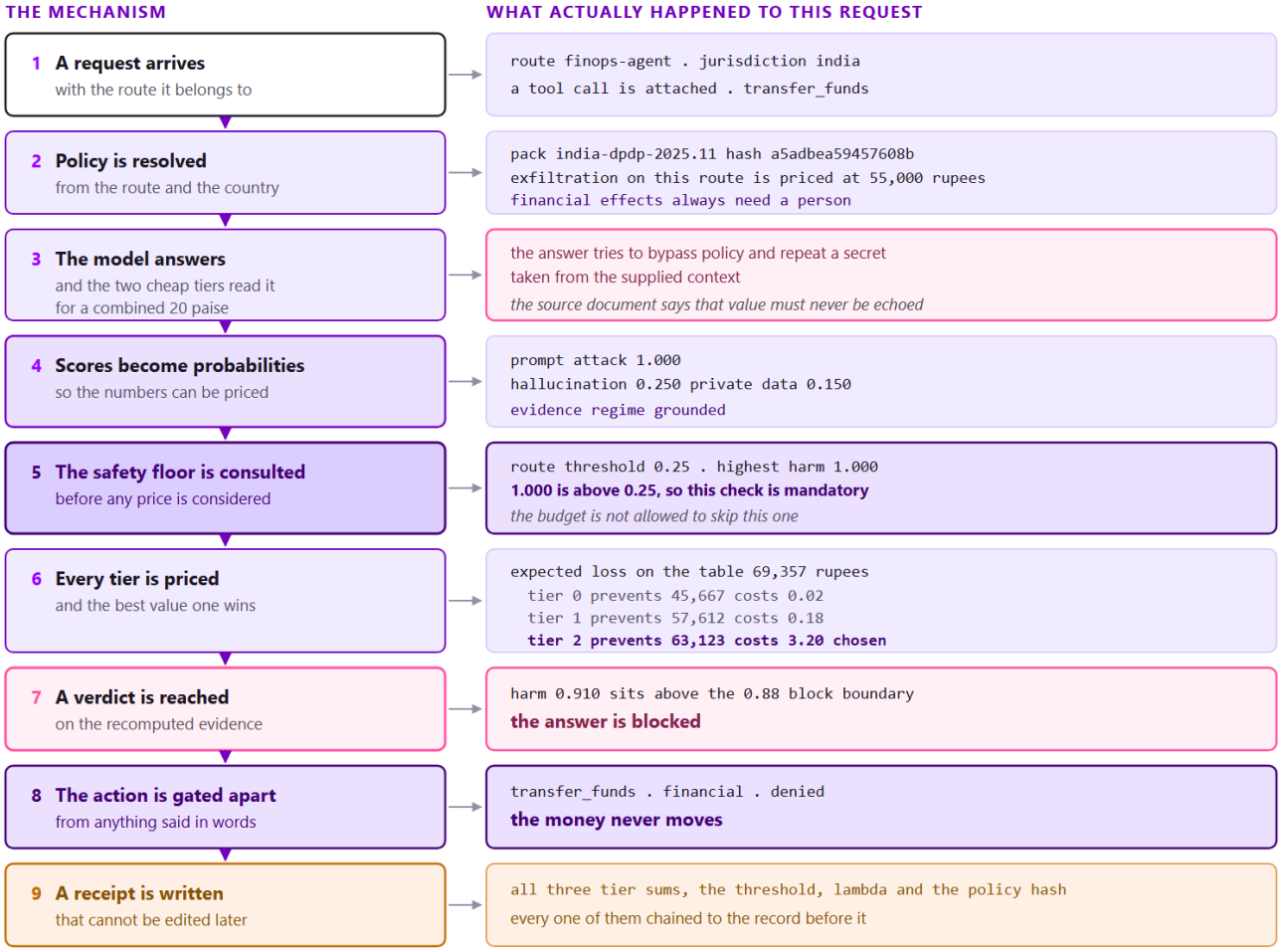
The model answers. Two cheap tiers read the answer for a combined 20 paise and score it on five axes. Those scores are merged and calibrated into probabilities.

The safety floor is consulted first. If the highest calibrated risk sits at or above the route threshold, a check is obligatory. Only then does the allocator price each tier and pick the one with the best net value. If it buys the expensive judge, the judge runs and the scores are recomputed with its signal included.

A verdict follows from the calibrated numbers. Proposed actions are permitted, held or denied on their own. The whole decision is appended to the ledger, and anything the system declined to decide alone is queued for a person.

One real request, traced from arrival to receipt

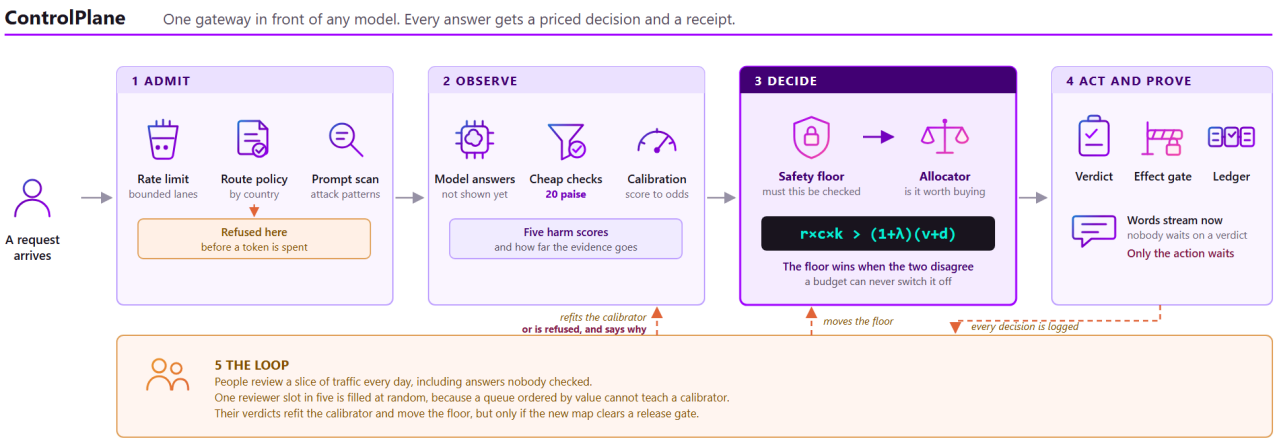
Held out row cp-02477. A finance route request carrying a transfer, where the model echoes a secret it was told never to repeat.



Stages five and six are the whole product. The floor decides whether a check is optional. The allocator decides which check is worth buying. Every number above is recomputed from the stored record, so the arithmetic behind this decision can be checked long after it was made.

Architecture

The gateway sees only the request, the response, any supplied context and any proposed tool calls. It needs no model weights, no hidden states and no log probabilities, so it works in front of a model you rent as well as one you host.



Stages one to four run on the request. Stage five runs overnight.
The gateway sees only what went in and what came out, so it works in front of a model an enterprise rents rather than owns.

Text is never delayed. Money and data are gated separately, and every decision is recomputable from its own record.

Plane	What it does
Admit	A per route token bucket and bounded lanes decide whether there is capacity. A preflight rules scan then reads the prompt alone and refuses it above an injection score of 0.70. Both happen before the model generates anything.
Observe	Tier 0 rules and Tier 1 signals score the answer on five harm axes. Isotonic calibration turns those scores into probabilities, and the evidence regime records what can be checked at all.
Decide	The release floor marks what must be checked. The allocator then prices each tier against the budget shadow price. Tier 2 runs only when selected, and the decision is recomputed with its signal.
Act and prove	A verdict of allow, annotate, abstain, hold or block covers the text. Proposed effects are permitted, held or denied independently. Everything is appended to the hash chain.
The loop	Reviewers label a slice of traffic every day. One reviewer slot in five is filled at random, because a queue ordered by value cannot teach a calibrator. Those labels refit the calibration maps, but only if the new map clears a release gate.

The decision rule is small enough to read.

```
expected loss = calibrated risk × consequence
check when  expected loss × catch rate  >  (1 + shadow price) × check cost
```

Both diagrams are hand authored SVG that regenerate from their own source, so a figure cannot drift away from the system it describes. Component contracts and the full request sequence are in architecture notes.

Key features

Feature	Where it lives
Budget aware assurance	Expected loss against priced cost, with a shadow price that tracks live spend. `economics/allocator.py`
Route specific safety floor	A release threshold selected on a held out fold with a finite sample bound. `guarantees/conformal.py`
Tiered verification	Three tiers at 1x, 9x and 160x the cheapest cost, behind one adapter contract. `detectors/`
Human review prioritisation	A queue that serves by expected loss per reviewer minute against the deadline. `review/queue.py`
Effect gating	Actions permitted, held or denied independently of the text verdict. `effects/effect_gate.py`
Audit ledger	Hash chained decision and review records that verify as one chain. `ledger/`
Admission control	Bounded lanes with capacity reserved for mandatory checks. `runtime/admission.py`
Learning loop	Refits calibration from reviewer labels, and refuses to release a bad refit. `learning/refit.py`
Evaluation	Every published figure regenerates from a committed command. `eval/`
Console and prototype	An inspection console over the evidence, and a live two panel prototype. `console/`

Evidence

Everything below runs offline against a seeded provider and lexical detector stubs. No API key, no network call and no GPU. Freezing detector quality is what makes the measured gain attributable to allocation rather than to a vendor model, and it is why anyone can reproduce these numbers exactly.

Question	Result
Does allocation beat checking everything?	At a 10% budget it averts **₹5,078,000** for **₹480.98** of compute, against **₹5,469,400** for **₹4,800**. That is **92.8% of the benefit for 10.0% of the compute**, over 1,500 held out interactions.
Does the release floor hold?	Yes. Observed unchecked harm **0.0560, 0.0714 and 0.0529** by route against a bound of **0.15**, over **339, 476 and 397** released rows.
Are the risk scores calibrated?	Expected calibration error **0.008 to 0.043** by route.
Is the audit trail complete?	**1,500 of 1,500** decisions and **205** reviews in one valid chain, with **224 of 224** proposed effects logged.
Does review order matter?	Yes. From the same **166** completed reviews the shipped queue rule serves **1.59x** the expected loss that first in first out does, breaching 65 deadlines against 139 and shedding 2 high value cases against 15.
Do the money assumptions move decisions?	Across a **0.25x to 4x** band on every consequence price, **10.9%** of tier choices change and **0%** of verdicts flip.
Is the detector catch rate measured or assumed?	Measured. Tier 2 catches **0.930** against **0.880** configured, over **365** labelled observations.
Does spend stay inside the budget?	Yes once governed. Spend lands at **1.00x to 1.03x** of budget across the grid, against up to **3.75x** before.
Does it hold up under load?	Bounded admission cuts text p99 from **540.6 ms to 26.2 ms** and effect p99 from **1508.1 ms to 104.1 ms** at 400 offered requests per second, by refusing 499 requests rather than slowing everyone.

Three results are reported because they did not go our way.

- **Allocation is not universally better than a well tuned fixed rate policy.** At matched actual spend it wins 5 of 7 budgets on our corpus and only 1 of 7 on Aegis. It helps when the harm mix varies and blanket cheap coverage is unaffordable, and both conditions are computable before deployment. See allocation regime.
- **Aegis is a failure and is reported as one.** 0.811 AUPRC sits below the weakest published system on that benchmark. See Aegis.
- **The learning loop currently refuses on all three routes.** A single pass over 1,500 rows yields 73 usable labelled pairs against the roughly 100 per route the bound needs, so `make relearn` declines to release and says why. That is the gate working.

Deeper results live in summary, queue comparison, PII probe, sensitivity, runtime and external benchmarks.

Two things here are configured assumptions rather than measurements. The rupee consequence of each harm axis and the ₹120 reviewer cost come from `config/`, so every monetary figure inherits them. The sensitivity sweep exists to bound how much that matters.

Demo

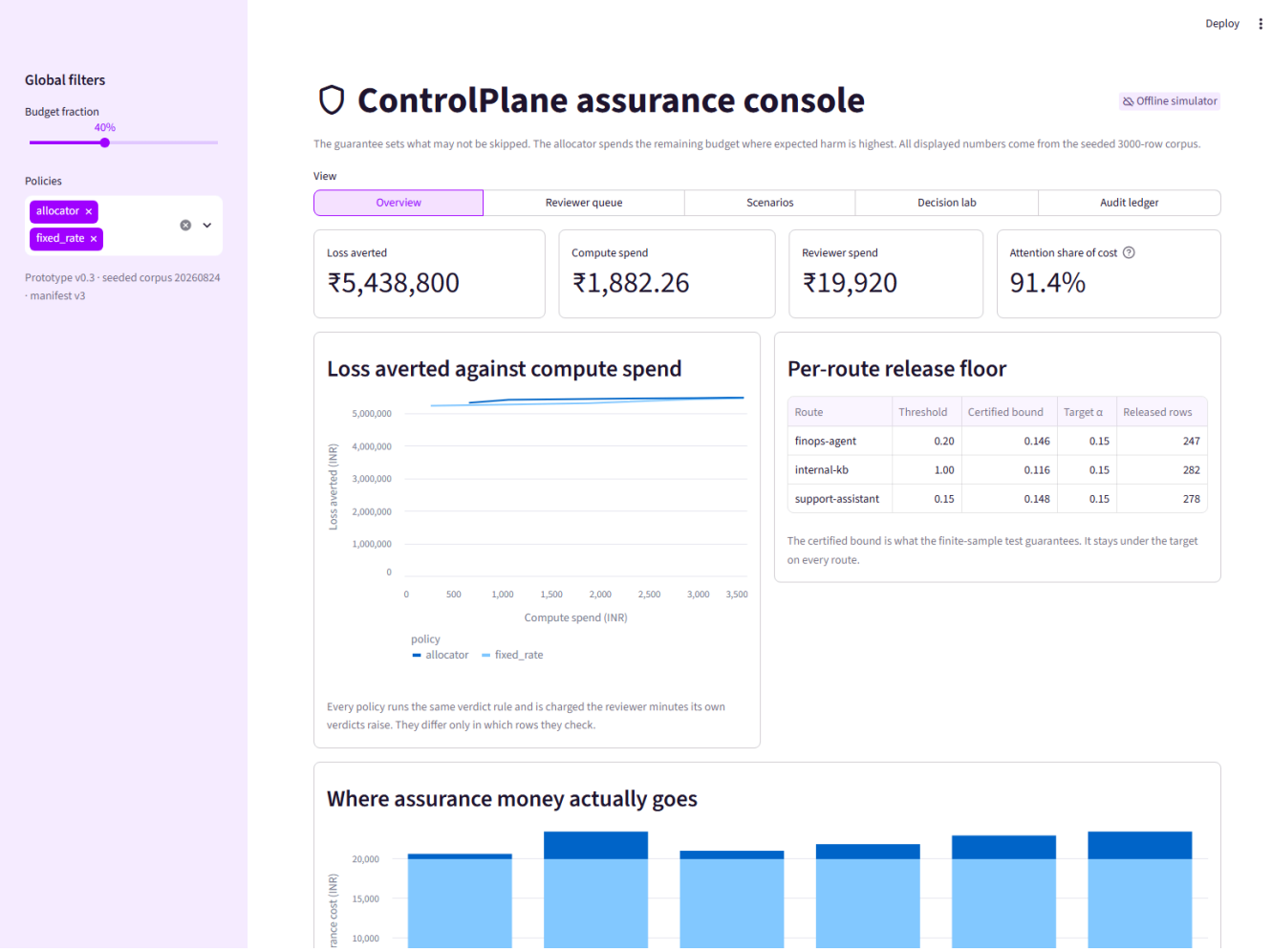
Two things are worth opening, and they show different halves of the project.

The live prototype puts a support chatbot next to the control plane. The customer view is on the left and every check, price and verdict is on the right, revealed one step at a time. Three preset requests show a prompt injection stopped before the model runs, an ordinary lookup the allocator declines to spend on, and a quiet data leak that the cheap tiers miss and the judge catches.

```
make prototype
```

The inspection console has six views over the committed evidence, including the allocation sweep, the release floor, calibration, the reviewer queue and the ledger. What each view shows and what to look at first is in the console guide.

```
make console
```



Requirements

- Python 3.11 or newer
- No API key, no network access and no GPU for the default path
- Roughly 500 MB of disk for the virtual environment and the generated corpus
- Optional. A Groq API key if you want the prototype to call a live model instead of a fixed reply
- Optional. Several GB of disk and a network connection to reproduce the external benchmark results

Installation

```
git clone https://github.com/Advait-Pathak9222/BurningShadows.git
cd BurningShadows
make install
```

On Windows PowerShell without `make`.

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
```

Quick Start

```
make demo        # builds the corpus, calibrates, runs the scenarios, verifies the chain
make console     # opens the inspection console at http://localhost:8501
```

`make demo` takes about 18 seconds and touches nothing outside the repository.

To run everything a reviewer would want in one command, from a clean clone.

```
./run_submission.sh
```

That creates its own virtual environment, runs the quality gate, regenerates the demo and the report, then checks that every regenerated artifact is byte identical to the committed one. It fails loudly if any published number stops reproducing.

On Windows PowerShell without `make`.

```
.\.venv\Scripts\python.exe -m controlplane.cli demo
.\.venv\Scripts\python.exe -m streamlit run console\streamlit_app.py
```

Configuration

Three files control behaviour. All are versioned and all are read at startup.

File	What it sets
`config/economics.yaml`	Tier costs, latencies and catch rates, the delay cost of each effect class, reviewer cost and capacity, the budget rate and the random review share
`config/policies/*.yaml`	Per route and per jurisdiction consequence prices, the release bound alpha, review deadlines, hourly budgets and which effect classes always need a person
`config/runtime.yaml`	Admission lanes, concurrency, queue depth and rate limits per route

The settings most worth understanding are these.

- **`consequence_inr`** in a policy file is what one instance of each harm costs on that route. It is the input every rupee figure depends on, and it needs finance sign off before deployment.
- **`alpha`** is the release bound. It is the share of released answers allowed to carry harm. The floor is only informative when alpha exceeds the harm base rate of your traffic.
- **`gateway_budget_rate_inr`** is assurance rupees per served interaction. The shadow price moves to hold it, the budget itself does not move.
- **`random_review_share`** is the fraction of reviewer slots filled uniformly at random. It costs queue performance and it is the only reason the calibrator can be refitted at all.

Repository Structure

Path	Purpose
`controlplane/gateway/`	OpenAI shaped API, streaming, and admission integration
`controlplane/runtime/`	Bounded concurrency, reserved mandatory capacity, load harnesses
`controlplane/detectors/`	Tiered detector interfaces, offline stubs, disclosure logic, optional adapters
`controlplane/risk/`	Per axis calibration and evidence regimes
`controlplane/guarantees/`	Per route finite sample release thresholds
`controlplane/economics/`	Cost model, budget controller, budget governor, allocator
`controlplane/review/`	Human review economics and queue strategies
`controlplane/learning/`	Refits calibration from reviewer labels, and the gate that can refuse one
`controlplane/effects/`	Independent effect gating
`controlplane/ledger/`	Hash chained decision and review records
`controlplane/eval/`	Reproducible evaluation, ablation, sensitivity and runtime commands
`controlplane/corpora/`	External benchmark loaders, each pinned to a commit and checksummed
`config/`	Versioned policies, economics and runtime limits
`data/`	Seeded synthetic calibration and held out traffic
`docs/results/`	Machine readable results and their written interpretations
`console/`	The inspection console and the live prototype
`site/`	Static project page
`tests/`	Invariants, failure behaviour, reproducibility and regression coverage
`run_submission.sh`	Clone and run reviewer entrypoint

Two files are worth reading before the rest. `allocator.py` holds the decision in plain arithmetic, and `conformal.py` holds the guarantee that overrides the budget.

Reproducing Results

Every published figure is written by a committed command into a tracked file.

Command	Writes
`make report`	`docs/results/summary.md`, allocation, floor, calibration and audit
`make attention`	`docs/results/attention.md`, the reviewer queue comparison
`make pii-probe`	`docs/results/pii.md`, disclosure detection and ablations
`make sensitivity`	`docs/results/sensitivity.md`, the consequence sweep
`make loadtest`	`docs/results/runtime.md`, admission control under load
`make relearn`	`data/learned/`, refits the calibrator or refuses and says why
`make toxicchat`	the ToxicChat probe, downloads the corpus
`make benchmarks`	`docs/results/benchmarks.md` plus Aegis, OR-Bench, BeaverTails and RAGTruth results, downloads the corpora

The two download targets fetch each corpus at a pinned commit revision and verify its SHA-256 before use, so a corpus re uploaded upstream fails loudly rather than silently changing what these numbers mean. No external dataset is vendored in this repository.

Testing

```
make check       # ruff, mypy --strict, and the full test suite
```

The suite covers decision invariants, failure behaviour, reproducibility and regression coverage of the specific defects this project has already found and fixed.

Deployment

The default path is offline by design, and offline is a property of the evidence rather than of the product. Tier 0 and Tier 1 belong in process because they run on all or most traffic. Tier 2 is where a hosted LLM judge belongs, and the detector contract is a single adapter method, so swapping one in does not touch the allocator. Where to draw that line, and what changes before an enterprise deployment, is set out in industry fit and deployment notes.

The console is deployed at controlplane-ai.streamlit.app.

Documentation

Document	What it covers
Architecture	Component contracts and the full request sequence
Industry fit	Where an LLM judge belongs and what offline does and does not mean
Deployment	Running the gateway and publishing the console
Console guide	What each console view shows and where its numbers come from
Assumptions	Every assumed input with its status and source
Limitations	The evaluation boundary and the evidence we would want next
Pre registration	What would have counted as success, written before each result
Business proposal	The commercial framing
Project handoff	Product story, architecture rationale and runbook in one place

Troubleshooting

`make` **is not available on Windows.** Use the PowerShell commands shown under Installation and Quick Start. Every target has a direct equivalent through `python -m controlplane.cli`.

Python version error from `run_submission.sh`**.** The script requires Python 3.11 or newer and reports the version it found. Point it at a newer interpreter with `PYTHON=/path/to/python3 ./run_submission.sh`.

The byte identity check fails. That means a committed artifact did not reproduce on your machine, which is a real finding rather than a nuisance. The script prints which files changed. Please report it with that list.

A benchmark download fails a checksum. The corpus was re uploaded upstream. The loader refuses to continue rather than silently changing what the published numbers mean. The pinned revisions are in `controlplane/corpora/`.

Port 8501 is already in use. Another Streamlit app is running. Stop it, or pass `--server.port 8502` to the `streamlit run` command.

The prototype shows a fixed reply instead of calling a model. No API key was found. Add a Groq key in the sidebar or set `GROQ_API_KEY`. The control plane behaves identically either way, because it only ever sees text.

Scope and assumptions

Loss and cost figures are arithmetic over synthetic traffic and configured consequence prices. They describe this implementation and its assumptions rather than a customer result. Tier 2 is a deterministic stand in for a production judge, calibration is fitted on synthetic traffic and would need refitting on real traffic, and the ledger and budget controller assume a single process. Every assumed input is listed with its status in assumptions, and the full evaluation boundary is in limitations.

Licence and data

The code and the generated synthetic corpus are MIT, see `LICENSE`.

No external dataset is vendored here. The five evaluation corpora are downloaded at run time into `data/external/`, which is untracked, each at a pinned commit revision whose SHA-256 is verified before use. They keep their own licences. RAGTruth is MIT, Aegis and OR-Bench are CC-BY-4.0, and ToxicChat and BeaverTails are CC-BY-NC-4.0. Results derived from the two non commercial corpora are reported for research and evaluation only, and no commercial claim in this repository rests on them alone.